

API TESTING & EXECUTION REPORT

Simple Books and Countries API Project

Assignment No. 4

Submitted By	Farooque Sajjad
Department	Quality Assurance
Organization	10Pearls
Submission Date	05-14-2026
Tools Used	Postman

1. Document Control

This document covers API testing design and execution using Postman for the Simple Books API project. The assignment demonstrates REST API testing using multiple HTTP methods, authentication handling, API chaining, variable management, and Chai assertions.

Version: 2.0

Prepared By: Farooque Sajjad

Status: Draft

2. Project Summary

The Simple Books API project is a REST API testing assignment executed in Postman. The collection validates API endpoints using GET, POST, PATCH, PUT, and DELETE methods. Dynamic variables, token-based authentication, request chaining, and automated assertions are implemented.

2.1 Scope of Testing

- API status validation
- Book retrieval APIs
- Authentication token generation
- Order creation and management
- CRUD operations on orders
- External API testing using REST Countries API
- Chai assertion validation
- Negative testing using intentional failures

2.2 Out of Scope

- Performance testing
- Load testing
- Security penetration testing
- UI testing

3. Test Methodology

3.1 Testing Approach

Black-box API testing was used throughout the assignment. All APIs were validated using request-response verification and automated assertions.

Testing Types Used:

- Functional Testing
- Smoke Testing
- Negative Testing

- Regression Testing

3.2 Tools Used

- Postman
- Postman Console
- REST APIs
- Chai Assertion Library

3.3 API Methods Covered

- GET
- POST
- PATCH
- PUT
- DELETE

4. Collection Variables and Environment Setup

The following collection variables were created and used for API chaining:

- baseUrl
- countriesBaseUrl
- accessToken
- orderId
- bookId
- clientName
- clientEmail

Variables were dynamically updated during execution using Postman collection variables.

5. API Requests and Execution Details

The collection contains 10 API requests covering complete CRUD workflow and external API testing.

5.1 GET - API Status Check

Purpose:

Verify API availability.

Assertions:

- Status code 200
- Status equals OK

- Response 1.65S

Intentional failing test was added for negative testing.

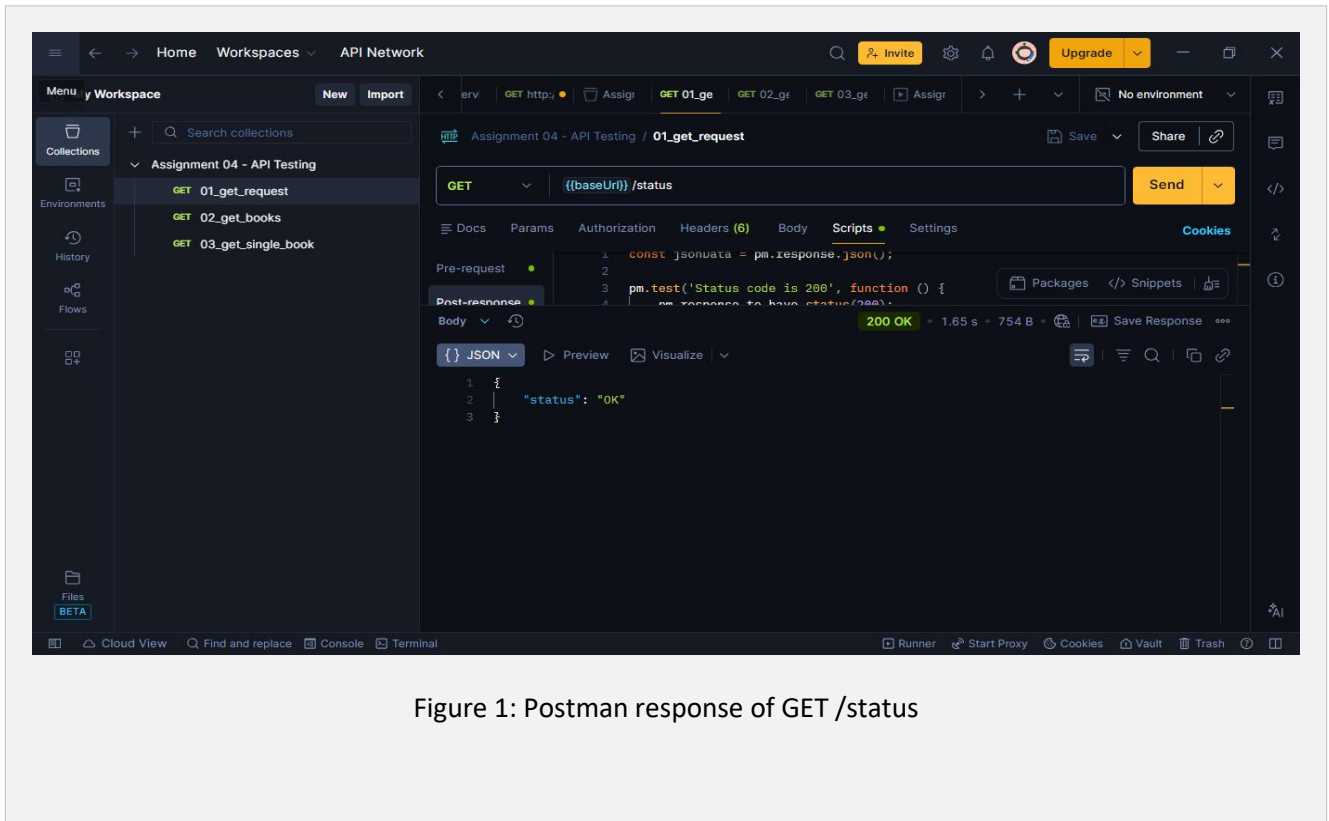


Figure 1: Postman response of GET /status

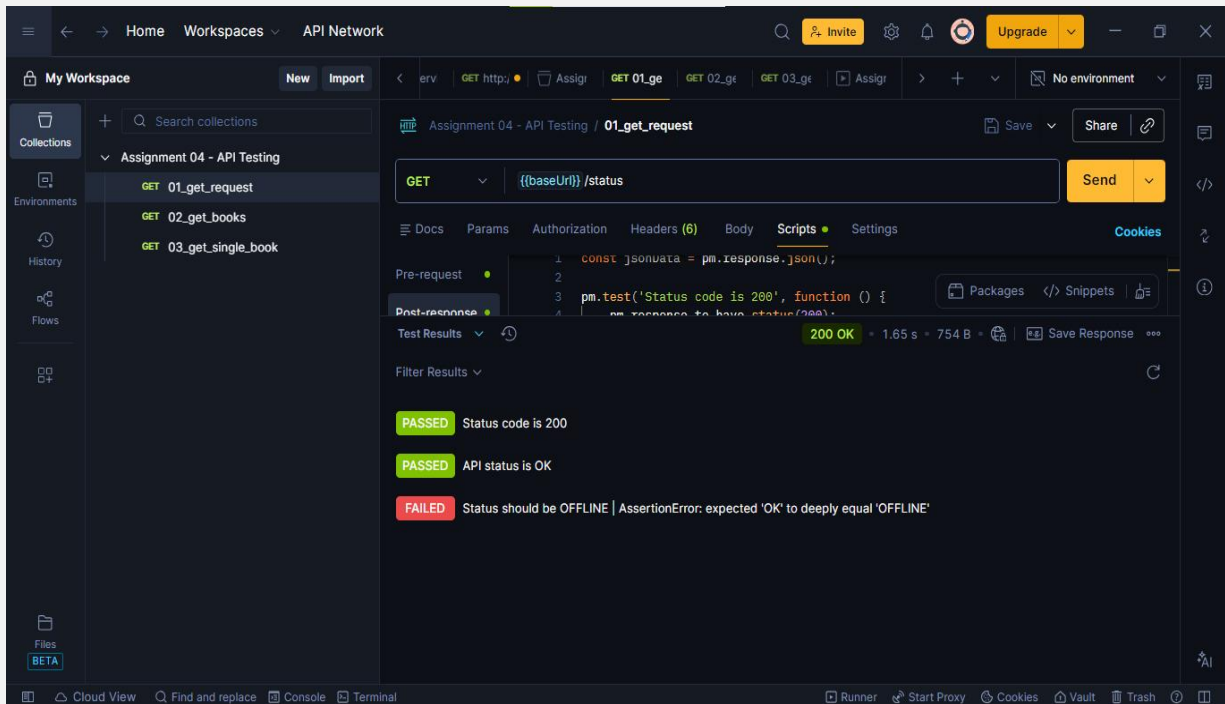


Figure 2: Failed assertion screenshot

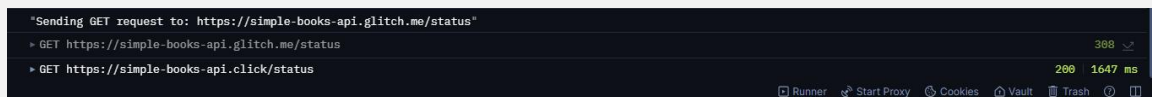


Figure 3: Postman Console log

5.2 GET - List Books

Purpose:

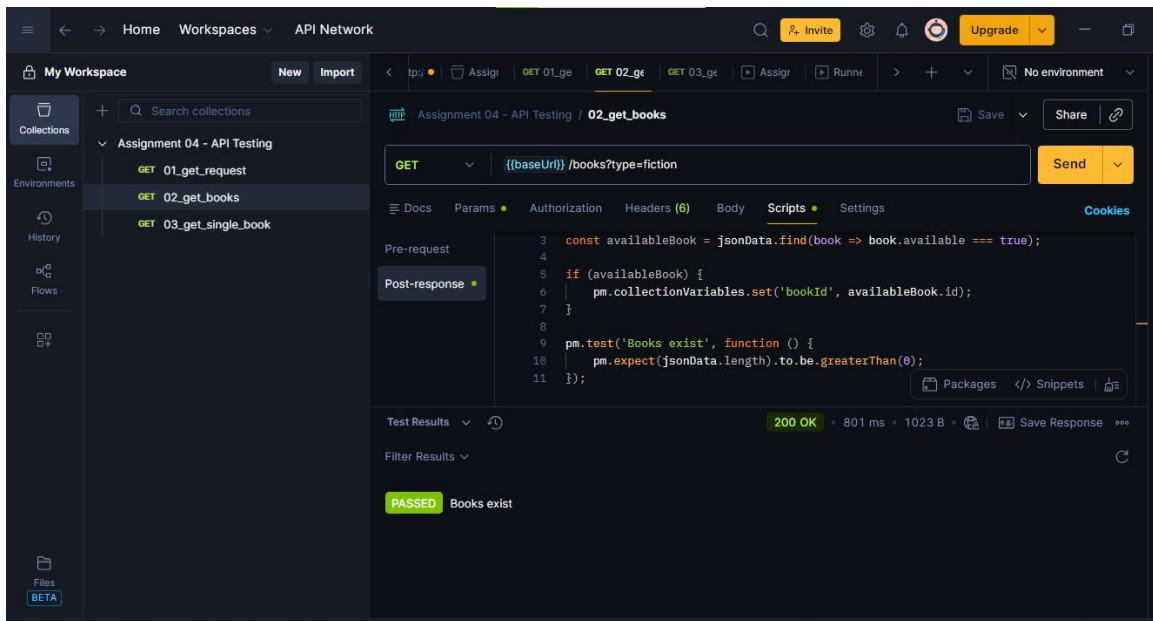
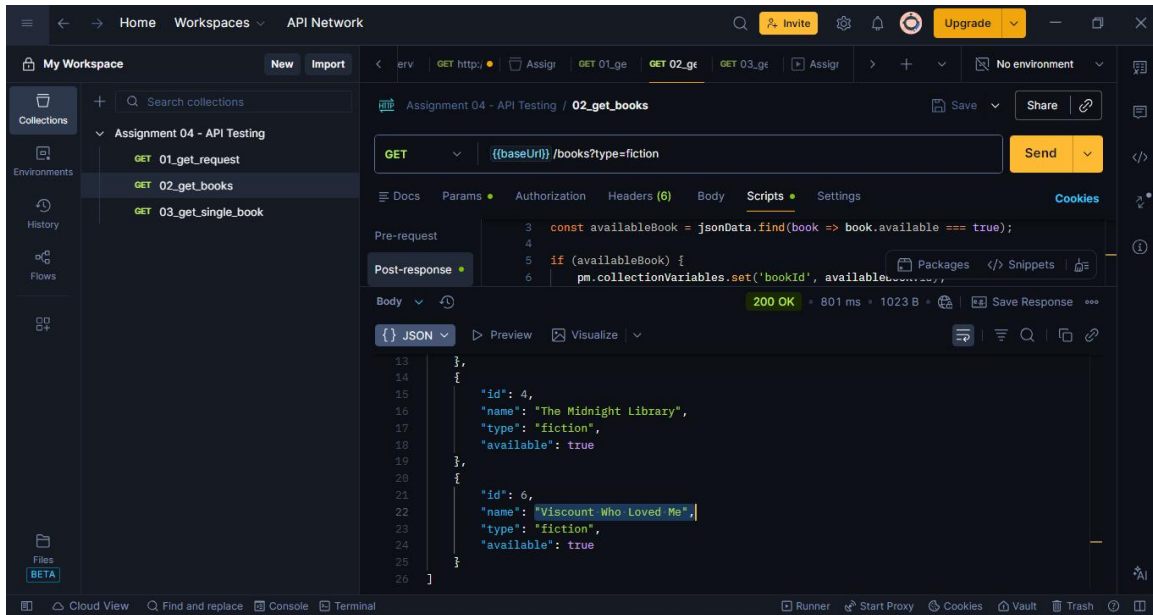
Fetch fiction books list and save bookId dynamically.

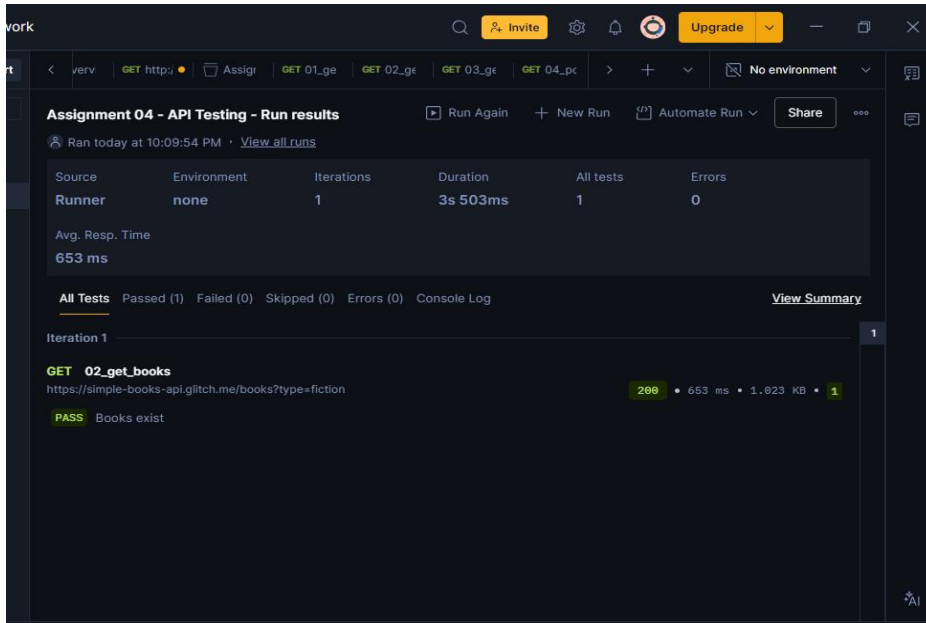
Features:

- Query parameter handling
- API chaining
- Dynamic variable storage

Assertions:

- Response is array
- Books list not empty
- Book fields validation





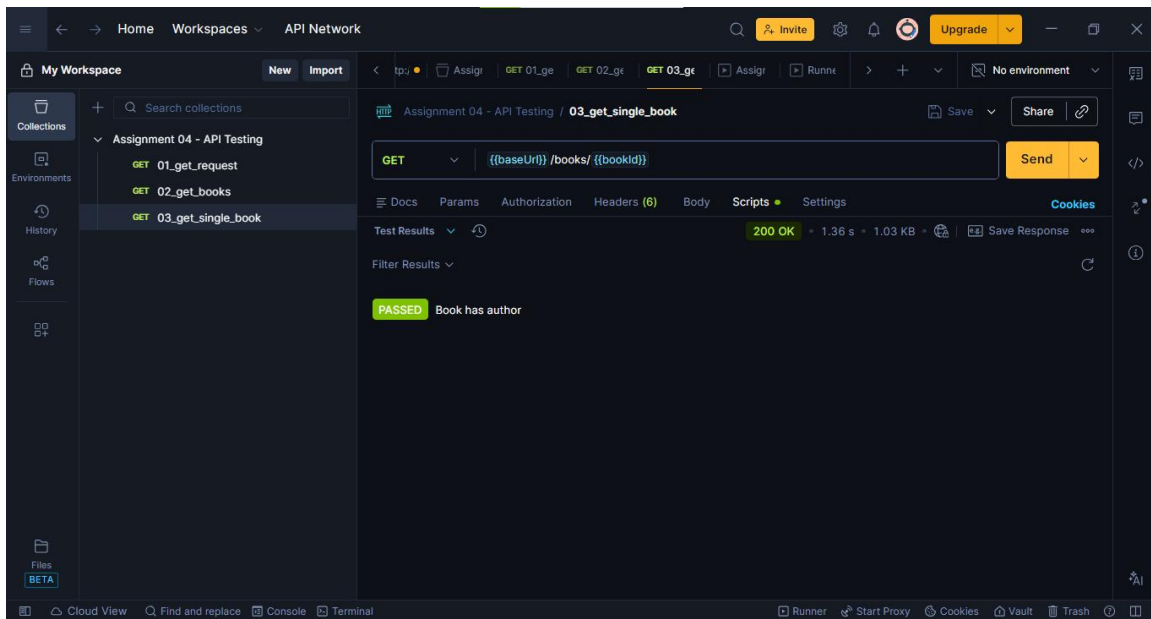
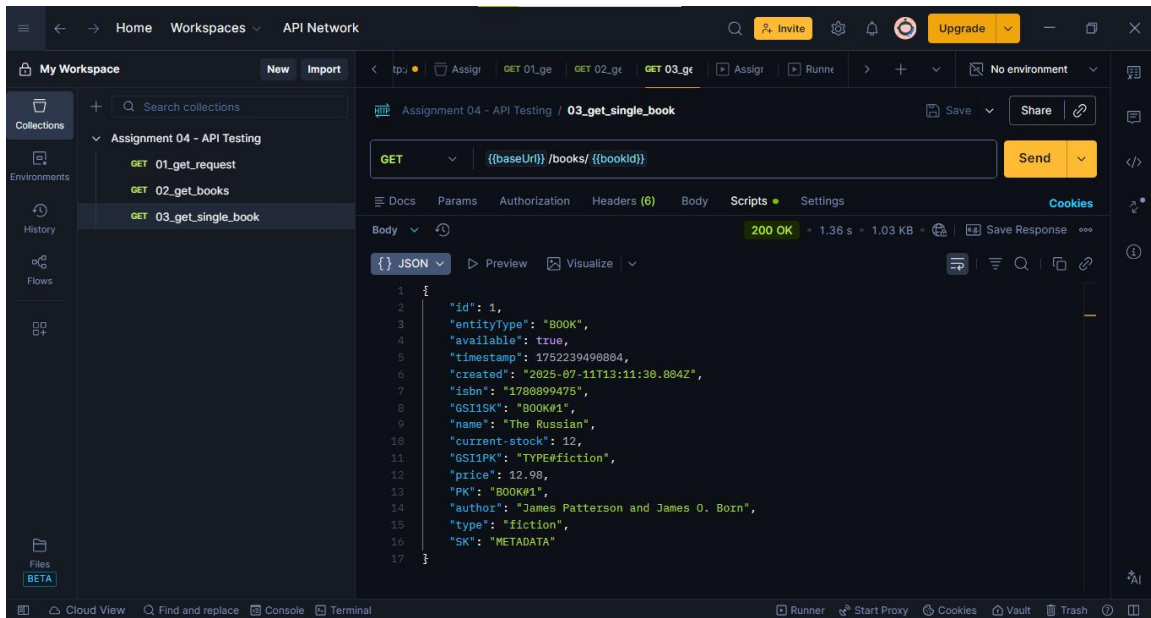
5.3 GET - Single Book

Purpose:

Retrieve a single book using chained bookId.

Assertions:

- Status code 200
- Book contains name and author
- Price validation
- Availability validation



5.4 POST - Register API Client

Purpose:

Register API client and generate access token.

Features:

- Random client generation

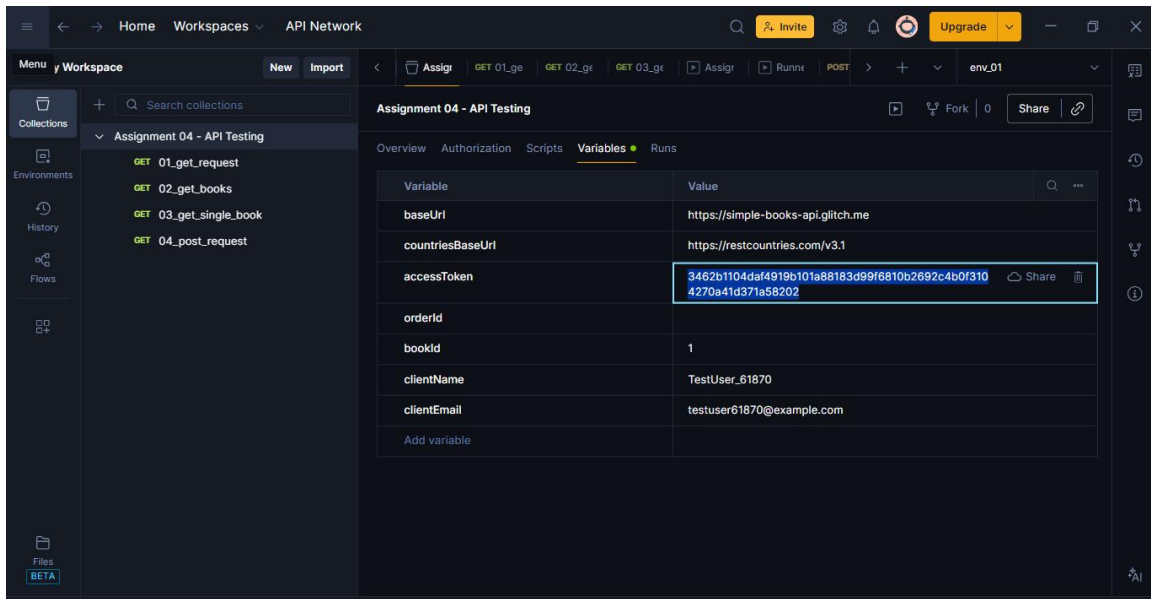
- Dynamic email creation
- Token storage

Assertions:

- Status code 201
- accessToken exists

The screenshot shows the Postman interface for a workspace named 'API Network'. The left sidebar shows a collection 'Assignment 04 - API Testing' with four requests: '01_get_request', '02_get_books', '03_get_single_book', and '04_post_request'. The '04_post_request' is selected and expanded. The request is a POST to '({baseUrl}) /api-clients'. The 'Pre-request' tab shows a script that sets a collection variable 'accessToken' to 'jsonData.accessToken'. The 'Post-response' tab shows a script that asserts 'Token exists' by checking if 'jsonData.accessToken' is not empty. The 'Body' tab shows a JSON response with a 201 status code, 585 ms response time, and 826 B body size. The JSON body is: { "accessToken": "3462b1104daf4919b101a88183d99f6810b2692c4b0f3104270a41d371a58202" }.

The screenshot shows the Postman interface for the same workspace and collection. The '04_post_request' is still selected, but the 'Headers' tab is now active. It shows a table with two headers: 'Content-Type' with value 'application/json' and an unchecked checkbox, and an empty row with a checked checkbox. Below the headers, the 'Test Results' section shows a 'PASSED' status for the 'Token exists' assertion. The response details remain the same: 201 Created, 585 ms, 826 B.



5.5 POST - Submit Order

Purpose:

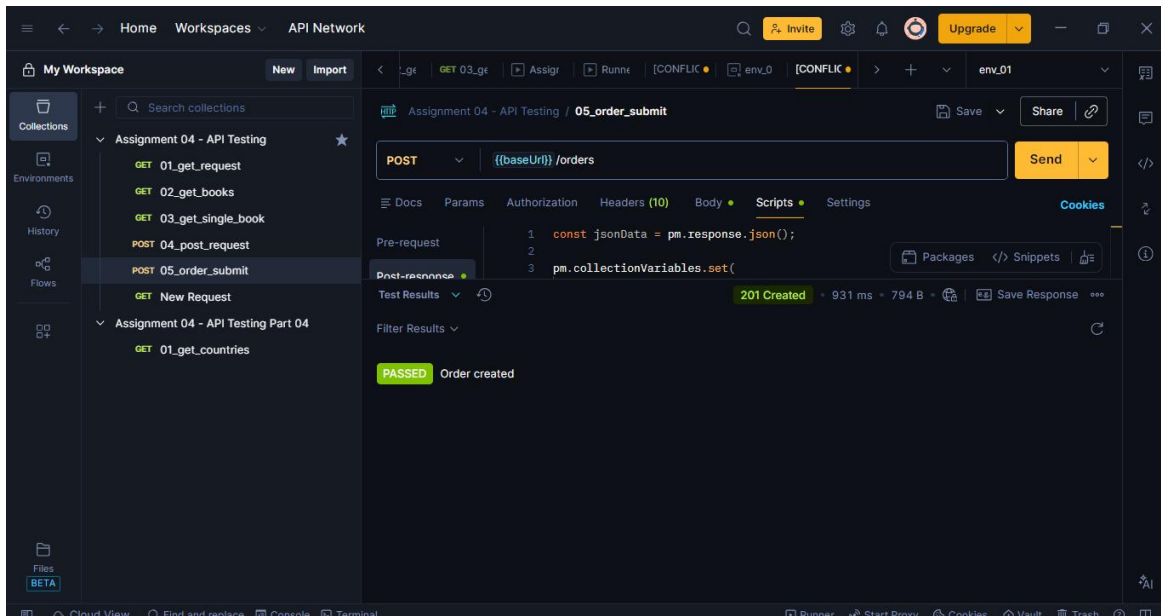
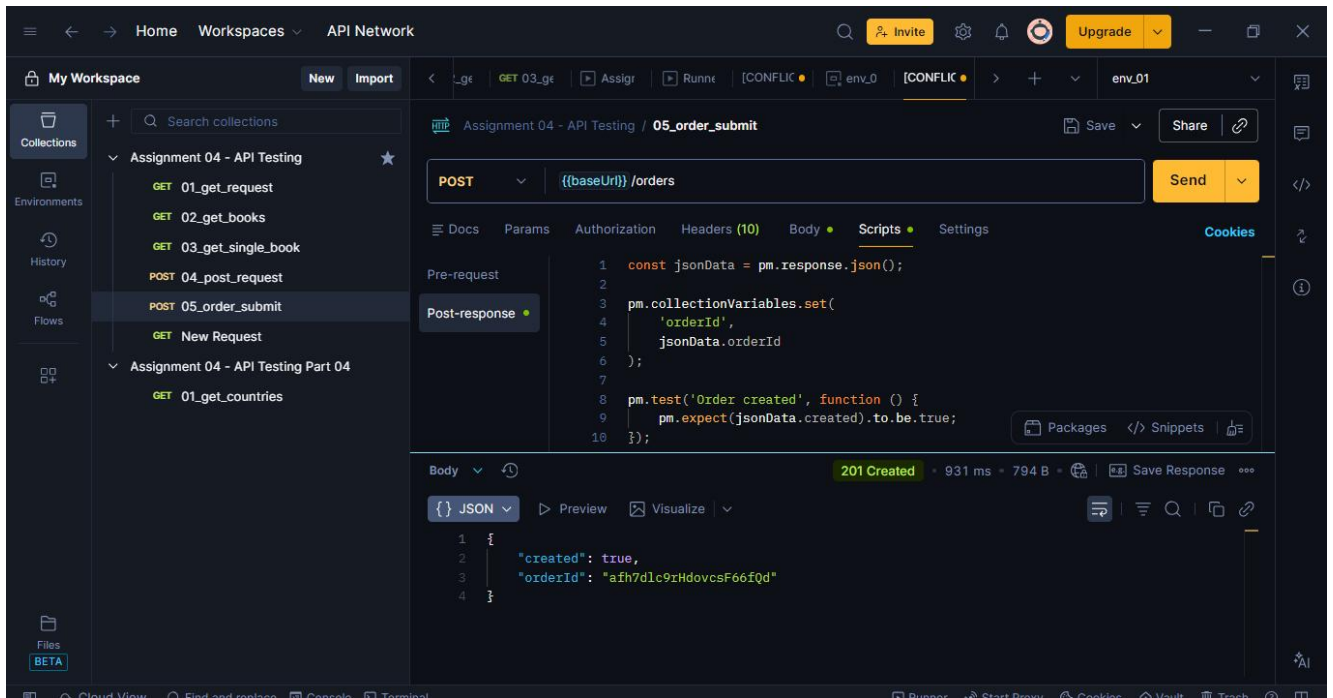
Create a new book order.

Features:

- Uses chained accessToken
- Uses chained bookId
- Dynamic customer names

Assertions:

- Order successfully created
- orderId generated



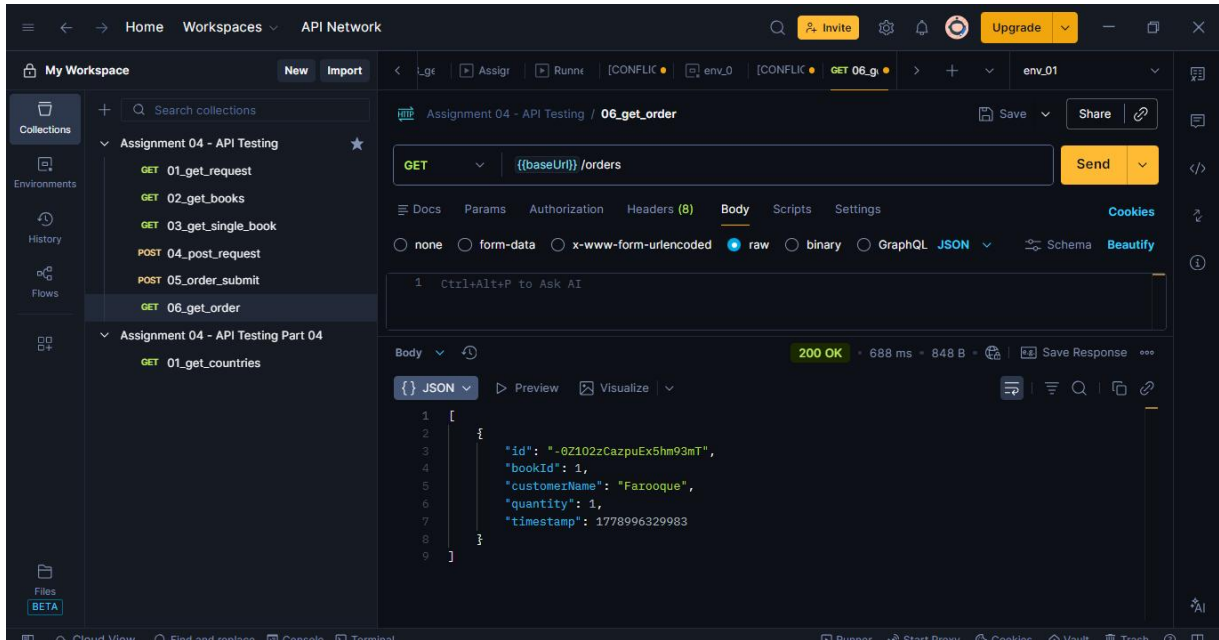
5.6 GET - Get All Orders

Purpose:

Retrieve all orders for authenticated client.

Assertions:

- Orders list validation
- Required fields verification



5.7 PATCH - Update Order

Purpose:

Update customer name partially.

Assertions:

- Status code 204
- Empty response validation

Home Workspaces API Network

My Workspace

Search collections

Assignment 04 - API Testing

- GET 01_get_request
- GET 02_get_books
- GET 03_get_single_book
- POST 04_post_request
- POST 05_order_submit
- GET 06_get_order
- POST 07_patch_request

Assignment 04 - API Testing Part 04

- GET 01_get_countries

Assignment 04 - API Testing / 07_patch_request

PATCH {{baseUrl}}/orders/{{orderId}}

Send

Docs Params Authorization Headers (10) Body Scripts Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body

204 No Content 1.02 s 746 B Save Response

{ } JSON Preview Visualize

1

Cloud View Find and replace Console Terminal Runner Start Proxy Cookies Vault Trash

Home Workspaces API Network

My Workspace

Search collections

Assignment 04 - API Testing

- GET 01_get_request
- GET 02_get_books
- GET 03_get_single_book
- POST 04_post_request
- POST 05_order_submit
- GET 06_get_order
- POST 07_patch_request

Assignment 04 - API Testing Part 04

- GET 01_get_countries

Assignment 04 - API Testing / 07_patch_request

PATCH {{baseUrl}}/orders/{{orderId}}

Send

Docs Params Authorization Headers (10) Body Scripts Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

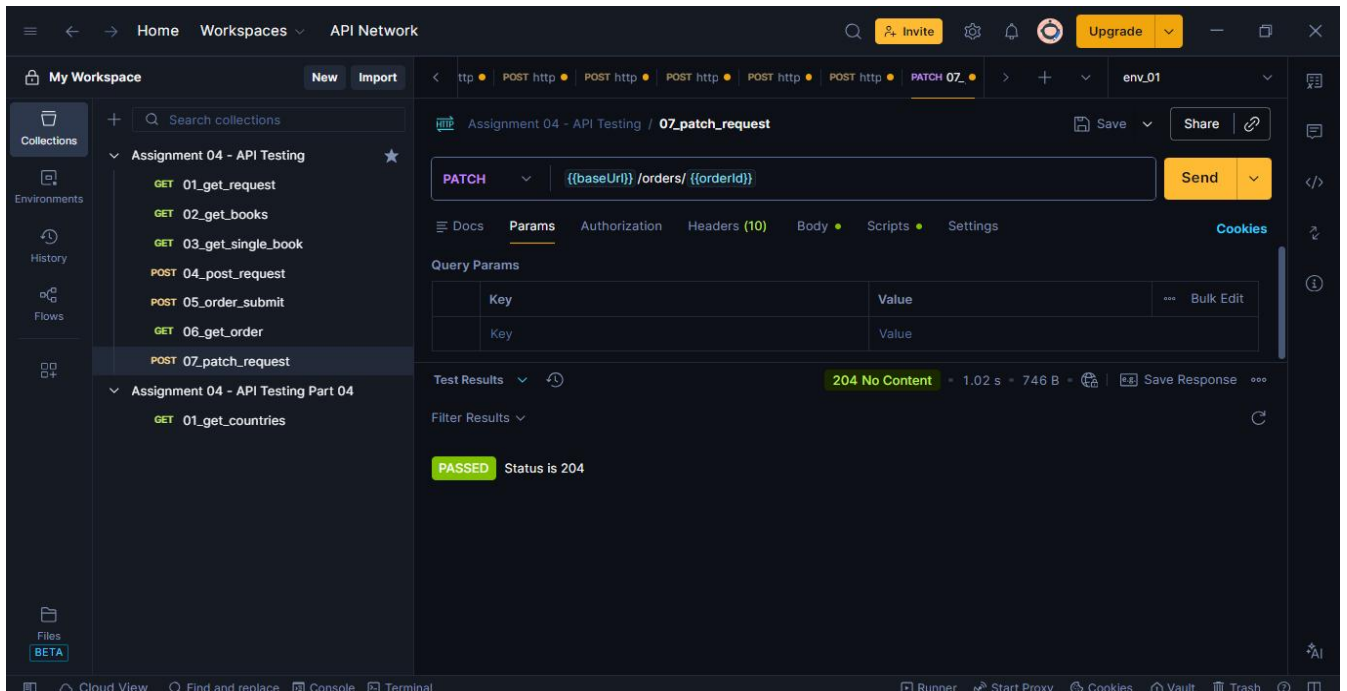
Body

204 No Content 1.02 s 746 B Save Response

{ } JSON Preview Visualize

1

Cloud View Find and replace Console Terminal Runner Start Proxy Cookies Vault Trash



5.8 PUT - Replace Order

Purpose:

Fully replace order data.

Assertions:

- Status code 204
- Empty response validation

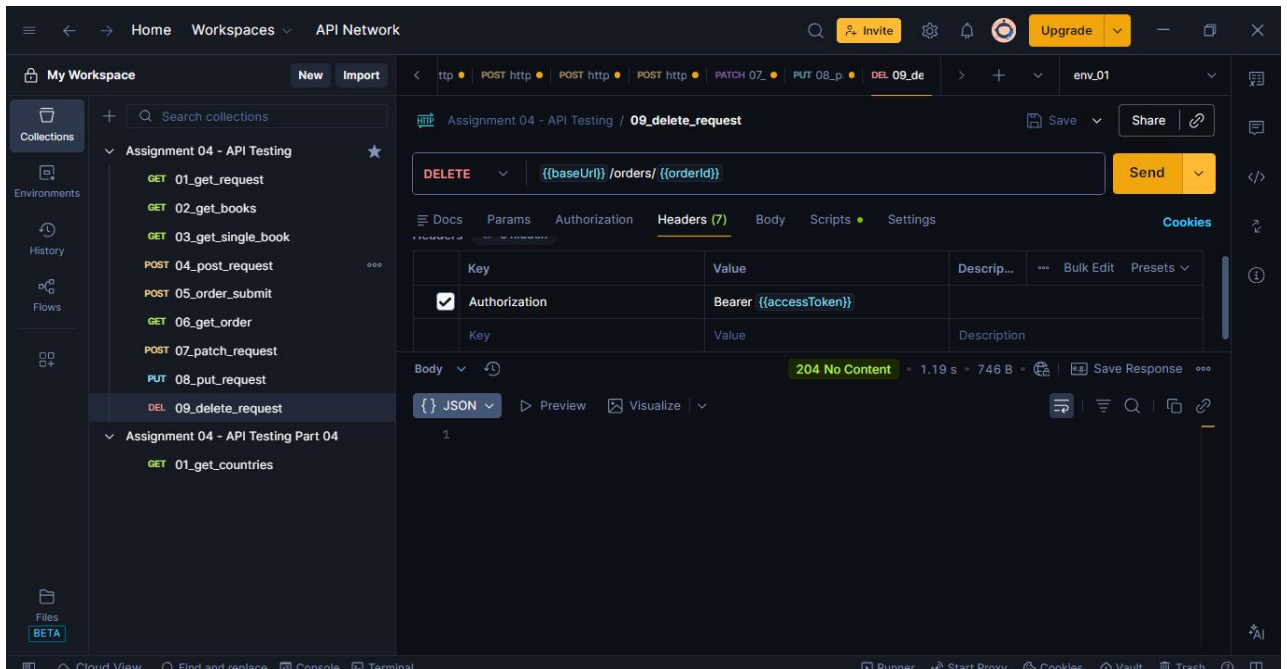
5.9 DELETE - Delete Order

Purpose:

Delete created order.

Assertions:

- Status code 204
- Empty response body



5.10 GET - Countries API

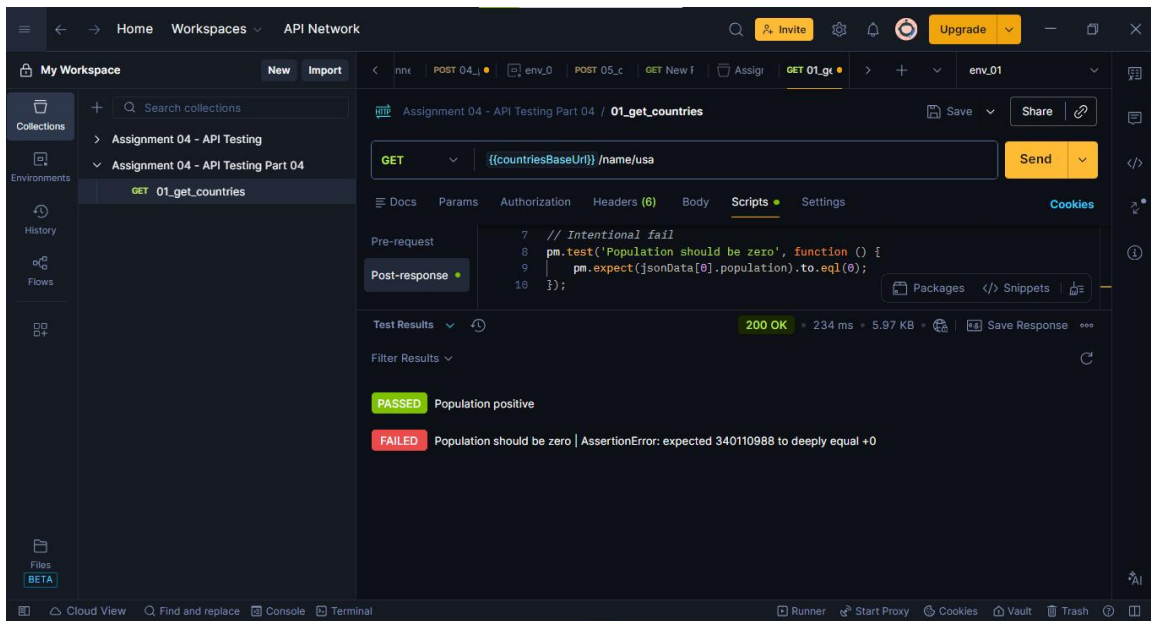
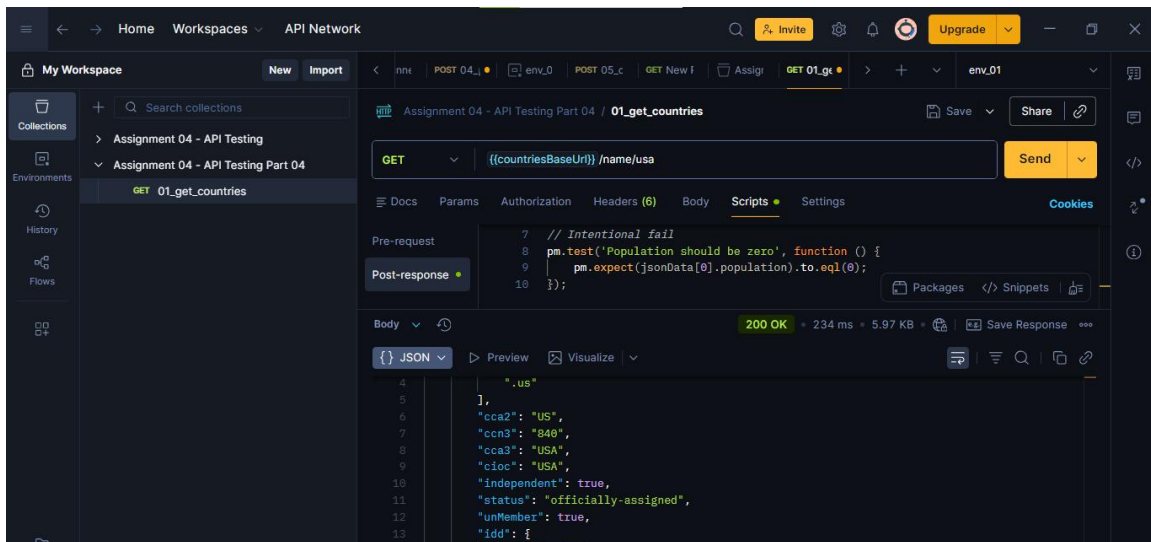
Purpose:

Validate external REST Countries API.

Assertions:

- Status code 200
- Population validation
- Country details verification

Intentional failing assertion added.



6. API Chaining Workflow

The following API chaining process was implemented:

1. Fetch books
2. Save bookId
3. Register API client
4. Save accessToken
5. Create order
6. Save orderId
7. Update order

8. Delete order

This demonstrates real-world API workflow automation.

7. Chai Assertions Used

Sample assertions used in the assignment:

```
pm.response.to.have.status(200);
```

```
pm.expect(jsonData).to.have.property('status');
```

```
pm.expect(jsonData.length).to.be.greaterThan(0);
```

```
pm.expect(jsonData.accessToken).to.be.a('string').and.not.empty;
```

```
pm.expect(pm.response.responseTime).to.be.below(2000);
```

8. Negative Testing

Intentional failing tests were added to demonstrate error reporting and validation failures.

Examples:

- Status should be OFFLINE
- Population should be zero

09. Observations and Learning Outcomes

The assignment provided practical experience in:

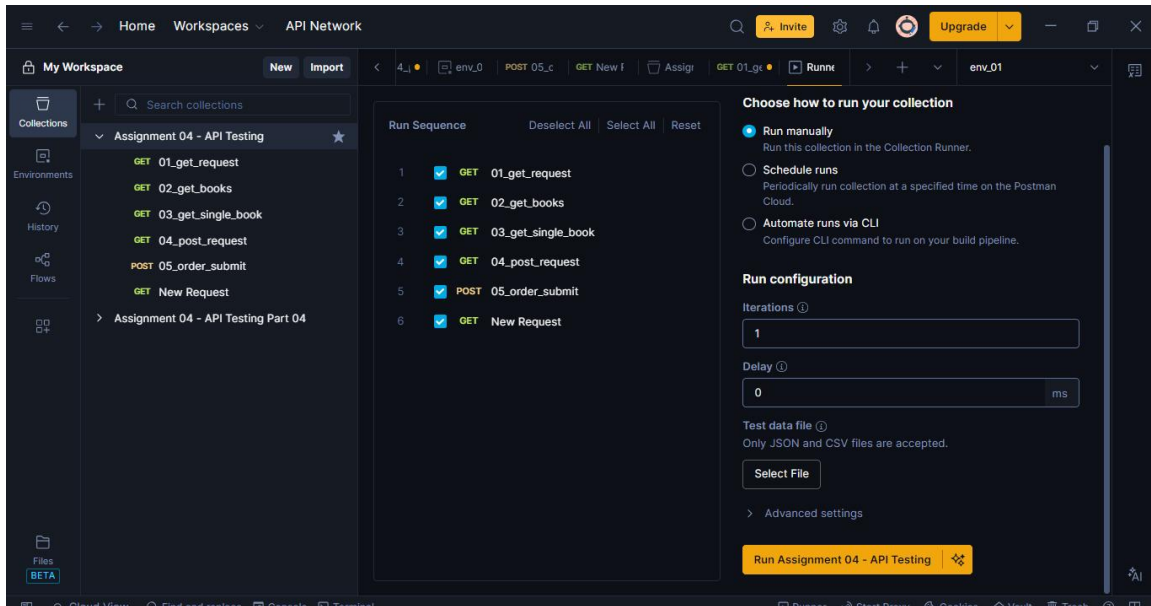
- REST API testing
- CRUD API operations
- Authentication handling
- Dynamic variables
- API chaining
- Chai assertions
- Automated validations
- Negative testing

- Debugging using Postman Console

The project demonstrates real-world API automation practices used in QA engineering.

11. Submission Checklist

1. Postman Collection JSON
2. Postman Screenshots
3. API Execution Report
4. Chai Assertion Examples
5. Console Log Screenshots
6. Collection Variables Screenshot
7. CRUD Workflow Screenshots



HomeWorkspacesAPI Network

4...env_0POST 05_cGET New IAssigGET 01_gcAssigenv_01

My WorkspaceNewImport

Search collections

Assignment 04 - API Testing

GET 01_get_request

GET 02_get_books

GET 03_get_single_book

GET 04_post_request

POST 05_order_submit

GET New Request

Assignment 04 - API Testing Part 04

FilesBETA

Assignment 04 - API Testing - Run resultsERROR

Ran today at 11:22:10 PM · View all runs

Debug with AIRun AgainNew RunAutomate RunShare

Source	Environment	Iterations	Duration	All tests	Errors
Runner	env_01	1	6s 954ms	5	1

Avg. Resp. Time
412 ms

RUN SUMMARYView Results

1

GET 01_get_request21X

PASSStatus code is 200

PASSAPI status is OK

FAILStatus should be OFFLINE

GET 02_get_books10

GET 03_get_single_book10

GET 04_post_request00